

Sub-Agent Architectures for Autonomous AI Systems

Principles, Patterns, and Research Applications

J.C. Vaught

University of South Carolina

jvaught@sc.edu

February 2, 2026

Abstract

Large language model (LLM) agents are increasingly deployed as autonomous systems capable of planning, tool use, and code generation. A single monolithic agent, however, faces hard limits: finite context windows, loss of focus on long tasks, and an inability to specialize. **Sub-agent architectures** address these constraints by decomposing work across multiple coordinated agents, each with its own context budget and domain focus. This document provides a comprehensive treatment of sub-agent design—from two-layer orchestration to hierarchical multi-tier systems—and examines how these patterns apply to academic research workflows including literature review, experimental design, data analysis, and manuscript preparation.

1. **Specialization.** Each agent focuses on a narrow domain, improving depth of reasoning.
2. **Parallelism.** Independent tasks execute simultaneously, yielding $2\text{--}8\times$ wall-clock speedups.
3. **Scalability.** Total available context scales linearly with the number of agents spawned.

This document proceeds as follows: Section 2 defines core terminology. Section 3 presents the foundational two-layer pattern. Section 4 extends to three-tier and hierarchical designs. Section 5 covers parallelization strategies. Section 6 details orchestration patterns. Section 7 maps these architectures onto research workflows. Section 8 provides implementation guidance, and Section 9 discusses limitations.

1 Introduction

Modern LLM-based coding agents such as Claude Code, OpenHands, and Aider operate within a single context window, typically 200K tokens. While impressive, this budget is consumed rapidly when an agent must simultaneously reason about strategy, manage state, read large codebases, and produce output. The result is a *context bottleneck*: as task complexity grows linearly, effective context availability shrinks super-linearly due to attention dilution and retrieval degradation.

Sub-agent architectures resolve this by splitting cognition across multiple independent agents, each receiving a fresh context window. The benefits are threefold:

2 Fundamental Concepts

2.1 What Is a Sub-Agent?

A **sub-agent** is an LLM invocation spawned by a parent agent to perform a scoped piece of work. The sub-agent receives its own system prompt, context, and tool access. It executes autonomously and returns a structured result to its parent. Critically, the sub-agent’s context window is *independent*—it does not consume tokens from the parent’s budget.

2.2 Tasks vs. Subagents

Two distinct primitives exist for delegating work, and the choice between them shapes system behavior significantly.

Table 1: Comparison of task and subagent primitives.

Property	Task	Subagent
Lifetime	Ephemeral	Persistent
Context	200K, isolated	Fresh per turn
State	Stateless	Maintains memory
Concurrency	Up to 10	Typically 1–3
Ideal for	Parallel, independent work	Iterative, multi-turn work

Tasks are fire-and-forget: the parent issues a prompt, the task agent executes, and a result is returned. There is no back-and-forth. **Subagents** persist across multiple interactions, enabling iterative refinement—useful when the output of one step informs the next.

2.3 The Context Window Constraint

Every LLM call operates within a fixed context window C (e.g., 200K tokens). A single-agent approach must fit planning, code, tool output, and reasoning into one window:

$$C_{\text{effective}} = C - C_{\text{system}} - C_{\text{history}} - C_{\text{tools}}$$

As conversations grow, C_{history} dominates, leaving little room for new reasoning. Sub-agents reset this equation: each receives the full C budget, minus only the context explicitly passed to it.

3 Two-Layer Architecture

The simplest and most widely used sub-agent pattern consists of two layers: an **orchestrator** (the main agent) and one or more **workers** (task agents).

3.1 Orchestrator Responsibilities

The orchestrator follows a five-phase cycle:

1. **Understand:** Parse the user request and identify scope.
2. **Plan:** Decompose into atomic, independently executable tasks.
3. **Spawn:** Create workers with clear instructions, context files, and success criteria.
4. **Wait:** Collect results; handle timeouts and failures.

5. **Synthesize:** Merge worker outputs into a coherent final result.

3.2 Worker Lifecycle

Each worker follows a simpler four-step protocol:

1. **Understand:** Read the task instructions and provided context.
2. **Execute:** Perform the scoped work using available tools.
3. **Validate:** Check output against the success criteria.
4. **Report:** Return structured findings to the orchestrator.

The worker must stay within its assigned scope. It does not modify files outside its mandate, does not communicate with other workers, and does not escalate decisions—it either succeeds or reports failure.

3.3 Communication Flow

Information flows in two directions with strict filtering:

- **Downward** (orchestrator $\rightarrow$ worker): task definition, relevant context, constraints, output format specification.
- **Upward** (worker $\rightarrow$ orchestrator): outcome status, findings, confidence level, recommendations.

The orchestrator never sends its full state downward, and workers never send their full reasoning trace upward. This *information filtering* is essential to preserve context budgets.

3.4 Dependency Patterns

Not all tasks are independent. Three canonical dependency patterns govern how workers are scheduled:

Parallel. No dependencies—all tasks run simultaneously. This maximizes speedup.

Linear. Task B depends on A 's output, C depends on B , etc. Execution is sequential.

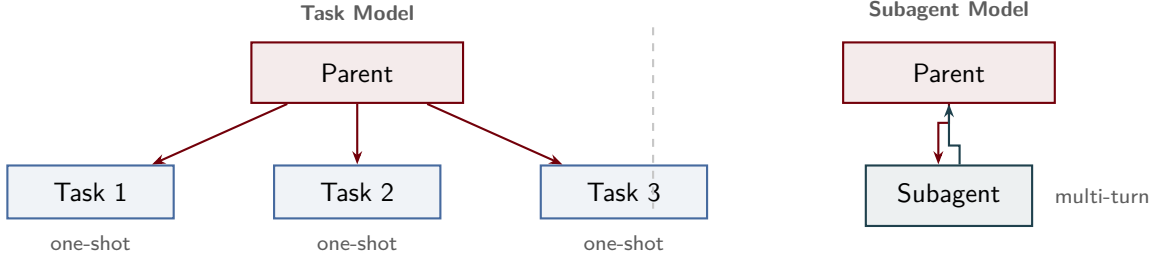


Figure 1: Task (left) vs. subagent (right) delegation. Tasks are one-shot and parallelizable; subagents support iterative dialogue.

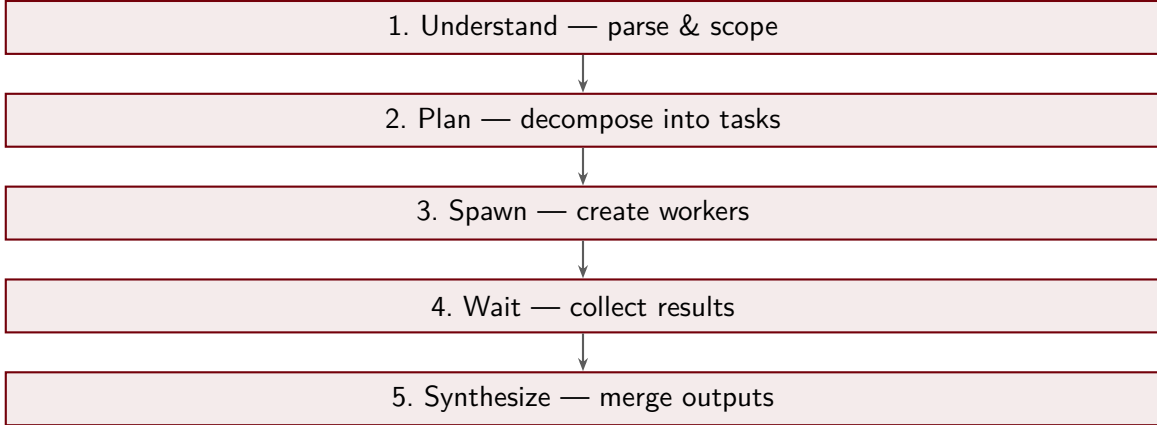


Figure 2: The five-phase orchestrator cycle.

Tree. Mixed: some tasks run in parallel, their outputs feed into dependent tasks. This is the most common real-world pattern.

3.5 Error Handling

Workers can fail for several reasons: timeout, out-of-memory, malformed output, or logical errors. The orchestrator applies graduated recovery:

1. **Retry** the task with a refined prompt or additional context.
2. **Decompose** further—the task may have been too broad.
3. **Partial results**—use what succeeded and note the gap.
4. **Escalate**—if multiple workers fail on the same task, flag a systemic issue to the user.

4 Multi-Tier Architectures

4.1 The Three-Tier Foundation

For complex projects, two layers are insufficient. The three-tier architecture inserts a **planning layer** between strategy and execution:

Why three tiers? The strategy layer reasons about *what* to accomplish; the planning layer determines *how* to decompose it; and the execution layer performs the actual work. This separation prevents the strategy agent from being overwhelmed by implementation details, and prevents execution agents from making strategic decisions they lack context for.

4.1.1 Token Budget Scaling

Total token consumption for a three-tier system with n execution agents:

$$T_{\text{total}} = T_{\text{strategy}} + T_{\text{planning}} + n \cdot T_{\text{execution}}$$

For $n = 5$ agents, a typical budget is $15\text{K} + 50\text{K} + 5 \times 35\text{K} = 240\text{K}$ tokens—far exceeding what a single 200K-context agent can achieve.

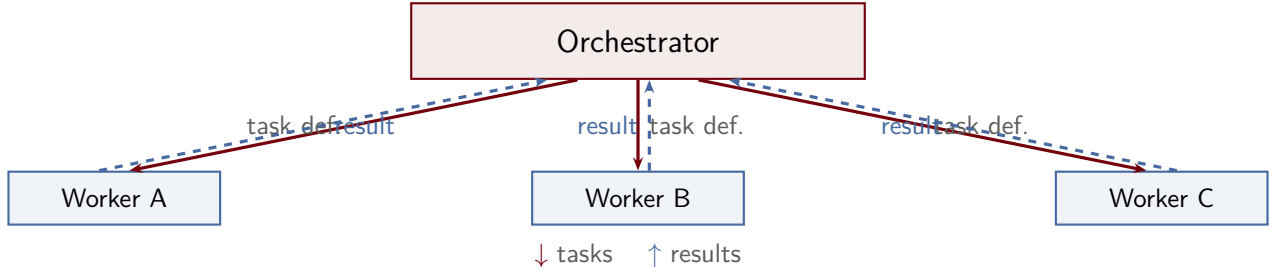


Figure 3: Two-layer communication: the orchestrator delegates tasks downward (solid) and receives results upward (dashed).

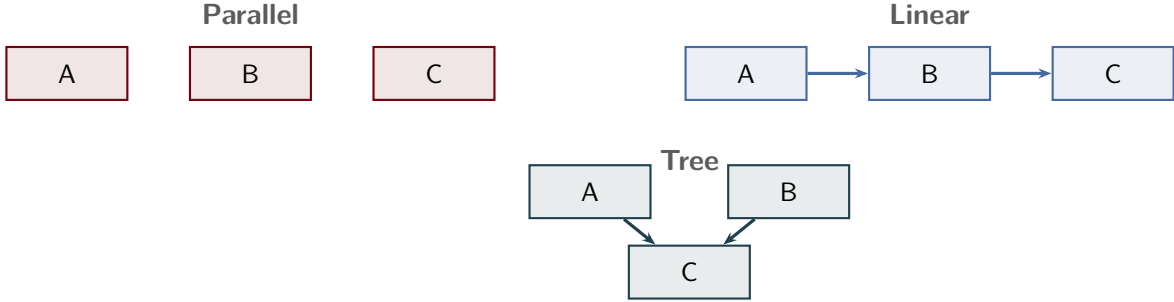


Figure 4: Three canonical dependency patterns for task scheduling.

4.2 Hierarchical Multi-Agent Systems

For very large problems, the three-tier model generalizes to an **arbitrary-depth hierarchy**. A root agent (the “chief architect”) delegates to domain leads, who in turn delegate to specialists.

4.2.1 Depth Limitations

Each level of nesting multiplies total token cost and adds latency. Empirically, the practical limit is **3–4 levels**:

Table 2: Token scaling with hierarchy depth (2 children per node).

Depth	Agents	Total Tokens
1	1	50K
2	3	100K
3	7	200K
4	15	400K
5	31	800K+ (overflow)

Beyond depth 4, coordination overhead, context contamination, and latency make deeper hierarchies impractical. Instead, use *flat delegation* (all coordination at one level) or *file-mediated pipelines* (Agent → File → Agent).

4.3 Leader-Worker Load Balancing

When many similar tasks must be processed, a **leader-worker pool** distributes work across a fixed or dynamic set of workers.

Common load-balancing strategies include round-robin (simple, fair), work-stealing queues (adaptive), priority queues (latency-optimized), and capacity-based assignment (balanced utilization).

5 Parallelization

5.1 Performance Impact

Parallelization delivers substantial improvements across task types:

The largest gains appear in data processing (60–70% faster), where tasks are naturally independent. Even tightly coupled tasks like code review see 35–45% improvement through domain-specialized parallel agents.

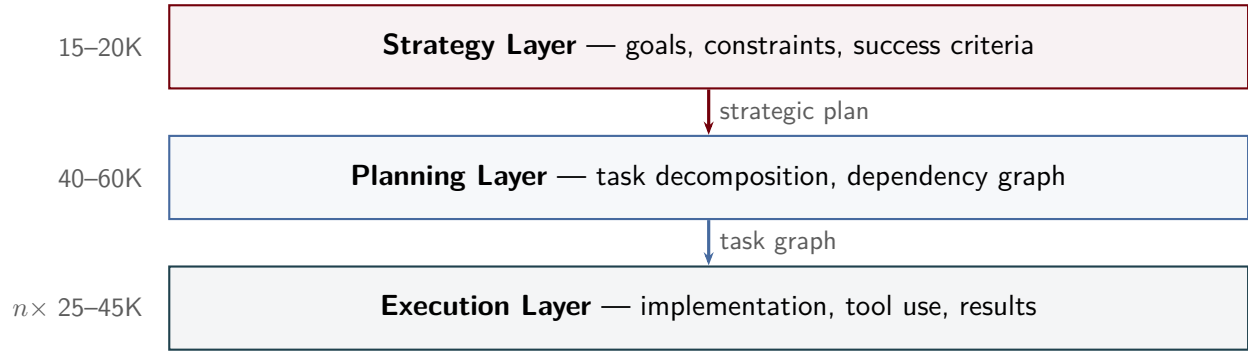


Figure 5: Three-tier architecture with typical token budgets per layer. The execution layer runs multiple agents in parallel.

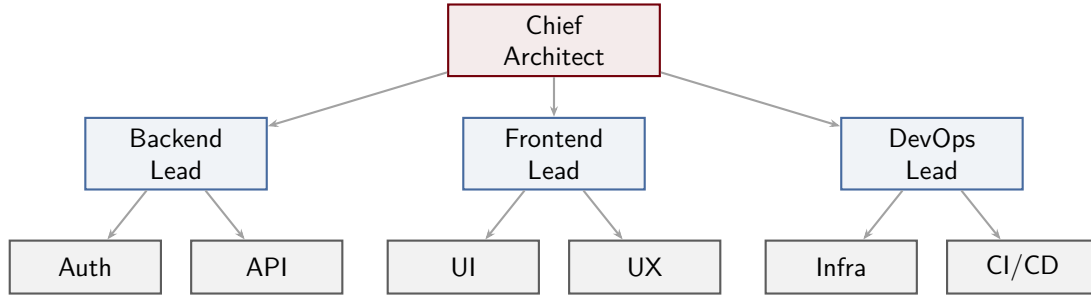


Figure 6: Hierarchical multi-agent system. Domain leads coordinate specialists; the root agent coordinates leads.

5.2 Cost-Speed Tradeoff

Parallelization is not free: each additional agent consumes its own token budget. The tradeoff is:

$$\text{Speedup} = \frac{T_{\text{sequential}}}{\max(T_{\text{agent}_1}, \dots, T_{\text{agent}_n})} \quad \text{Cost} = n \cdot T_{\text{tokens per agent}}$$

Parallelism is worthwhile when the time value of the speedup exceeds the marginal token cost.

5.3 Synchronization Barriers

When parallel agents must reconverge, three barrier types apply:

Full barrier. Wait for *all* agents. Used when every result is needed.

Partial barrier (k -of- n). Proceed when k of n agents finish. Used for redundant tasks (consensus).

Timeout barrier. Wait up to t seconds, then proceed with available results. Used for best-effort tasks.

5.4 Decision Framework

Table 3: When to use parallel vs. sequential execution.

Factor	Parallel	Sequential
Dependencies	None between tasks	Output feeds next task
Time	> 10 min sequential	< 5 min total
Quality	Multiple perspectives	Single expert suffices
Budget	Token-rich	Token-constrained
Coupling	Loosely coupled	Tightly coupled

6 Orchestration Patterns

Four canonical orchestration patterns recur across multi-agent systems. Each is suited to different problem structures.

Pipeline. Strict sequential handoff: Strategy → Planning → Execution₁ → Execution₂. Used when each stage transforms the output for the next.

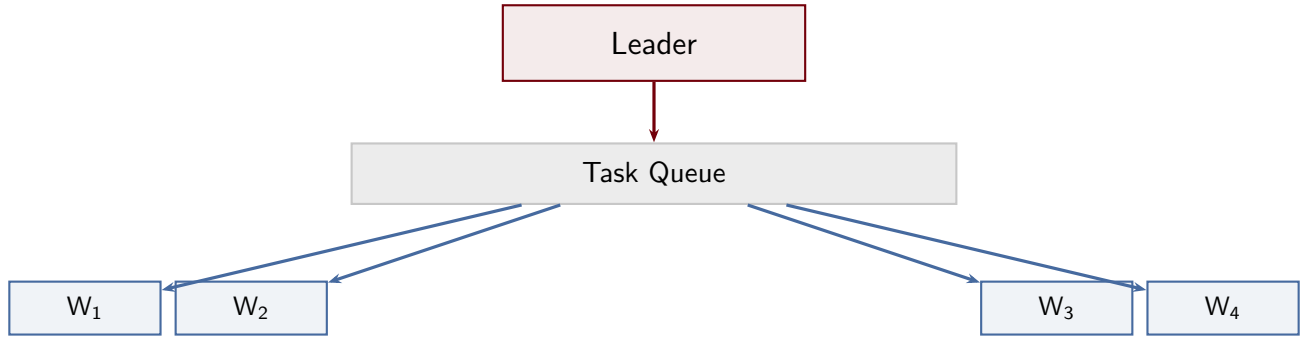


Figure 7: Leader-worker pool with a shared task queue. The leader enqueues tasks; workers dequeue and execute.



Figure 8: Wall-clock time comparison: sequential vs. parallel execution across four task categories.

Fan-Out/Fan-In. A planner distributes independent tasks to parallel workers, then a synthesizer merges results. This is the most common pattern for research tasks.

Master-Worker. A master repeatedly distributes similar tasks from a queue to a pool of identical workers. Ideal for batch processing (e.g., analyzing 50 files).

Feedback Loop. Execution results flow back to the planner for iterative refinement. Used in debugging, optimization, and any task where the first attempt is unlikely to be final.

6.1 Result Synthesis

When multiple workers return results, the orchestrator must merge them. The synthesis process follows five steps:

1. **Aggregate** all findings into a unified structure.
2. **Deduplicate** overlapping results.

3. **Prioritize** by severity, confidence, or relevance.
4. **Validate** consistency across workers (flag contradictions).
5. **Format** the merged result for the end user.

6.2 Conflict Resolution

When workers disagree, four resolution strategies are available:

- **Priority-based:** Higher-authority agent wins.
- **Vote-based:** Majority (k -of- n) consensus.
- **Consequence-based:** Evaluate downstream impact of each option.
- **Escalation:** Return the conflict to the user for a decision.

7 Research Applications

Sub-agent architectures map naturally onto the phases of academic research. This section demon-

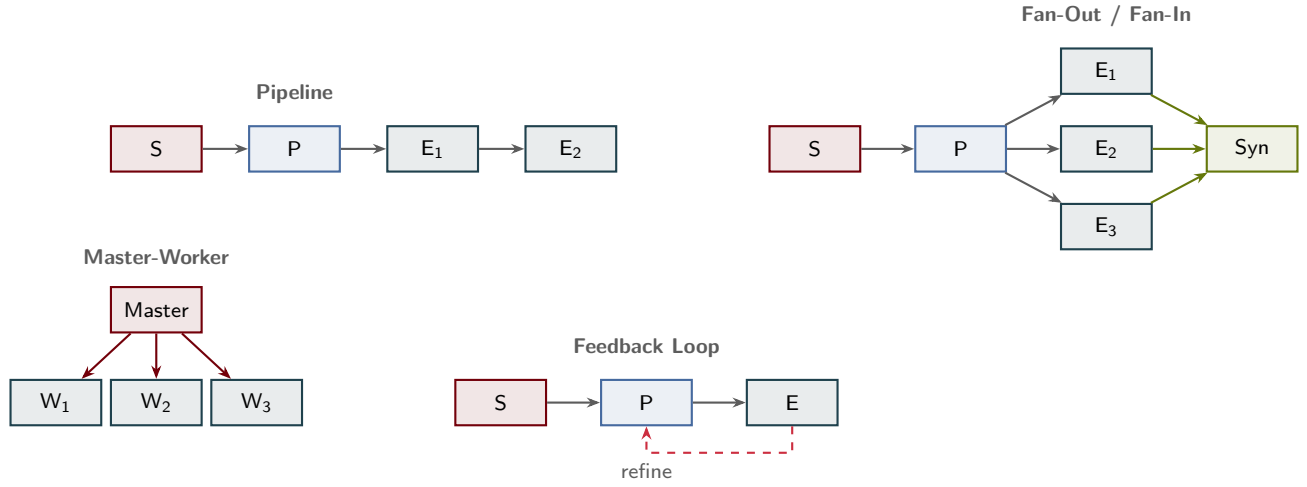


Figure 9: Four canonical orchestration patterns. S = Strategy, P = Planning, E = Execution, Syn = Synthesis. The feedback loop (bottom right) enables iterative refinement.

strates how each architectural pattern accelerates specific research activities.

7.1 Literature Review

A fan-out/fan-in pattern with three specialized agents:

- **Database Searcher:** Queries academic databases, retrieves abstracts, filters by relevance.
- **Citation Analyzer:** Maps citation networks, identifies seminal papers and emerging trends.
- **Gap Identifier:** Cross-references findings to identify unexplored areas and contradictions.
- **Synthesis Agent:** Merges all findings into a structured literature review with taxonomy.

7.2 Experimental Design and Execution

A three-tier architecture supports the full experimental lifecycle:

1. **Strategy agent** defines hypotheses, variables, and success metrics.
2. **Planning agent** designs the experimental matrix: parameter combinations, data splits, baseline configurations.
3. **Execution agents** (parallel) each run one experimental configuration, collect metrics, and report results.

For a typical hyperparameter sweep with 20 configurations, 10 parallel execution agents reduce wall-clock time by $\approx 50\%$ (2 batches of 10 vs. 20 sequential runs).

7.3 Data Analysis Pipeline

The master-worker pattern excels at processing large datasets:

Each worker analyzes its partition independently—computing statistics, generating visualizations, or running models. The aggregator then combines partial results into a global analysis.

7.4 Manuscript Preparation

A pipeline pattern supports the writing workflow:

1. **Outline Agent:** Structures the paper (sections, arguments, figure placement).
2. **Section Writers** (parallel): Each agent writes one section with access to relevant data and references.
3. **Figure Agent:** Generates plots, diagrams, and tables from experimental data.
4. **Integration Agent:** Merges sections, ensures narrative consistency, formats citations.
5. **Review Agent:** Checks for logical gaps, unsupported claims, and style issues.

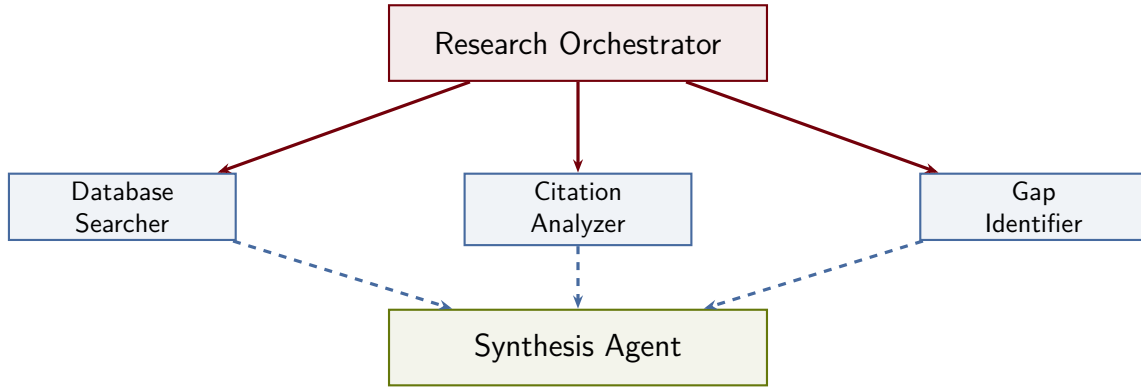


Figure 10: Literature review with three parallel agents feeding a synthesis agent.

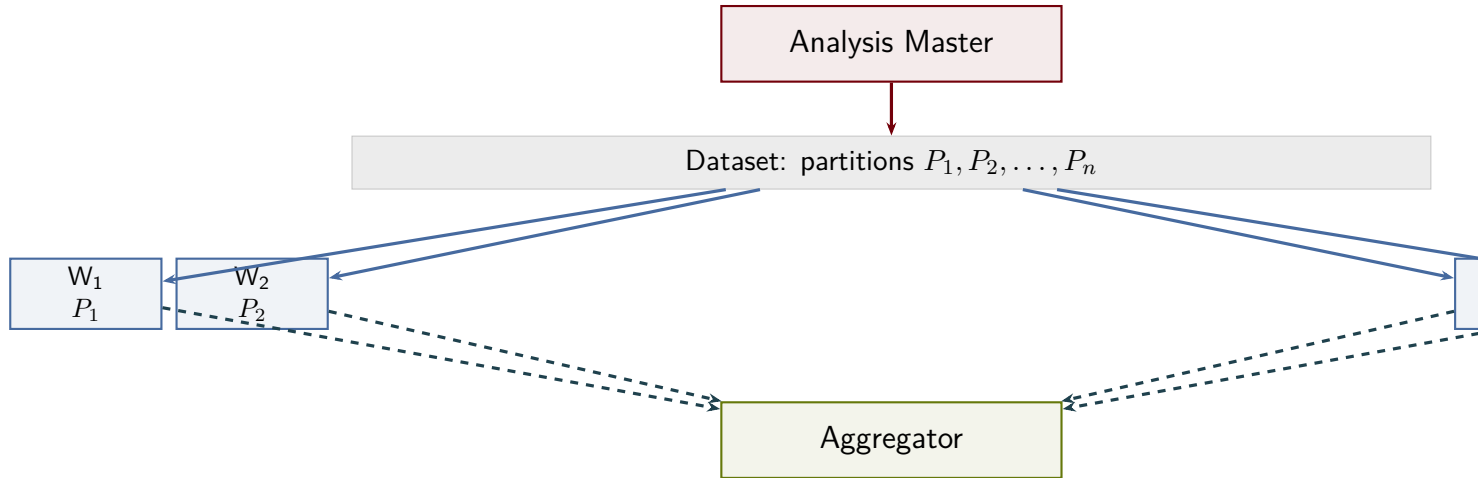


Figure 11: Data analysis pipeline: a master partitions the dataset and distributes to parallel workers. An aggregator merges results.

7.5 Full Research Workflow

Combining all patterns yields a four-phase research pipeline:

Table 4: Recommended agent roles for research workflows.

Role	Responsibility
Explorer	Codebase search, file discovery
Researcher	Literature search, reference gathering
Coder	Implementation, refactoring
Analyst	Statistical analysis, metrics computation
Visualizer	Plot generation, diagram creation
Reviewer	Quality checks, style enforcement
Writer	Prose generation, section drafting

8 Practical Implementation

8.1 Agent Specialization

Effective sub-agent systems define specialized roles. Table 4 lists common agent roles for research contexts.

8.2 Context Provisioning Best Practices

1. **Minimal context:** Pass only the files and information each worker needs—not the entire project.
2. **Explicit instructions:** Include success criteria, output format, and scope boundaries.

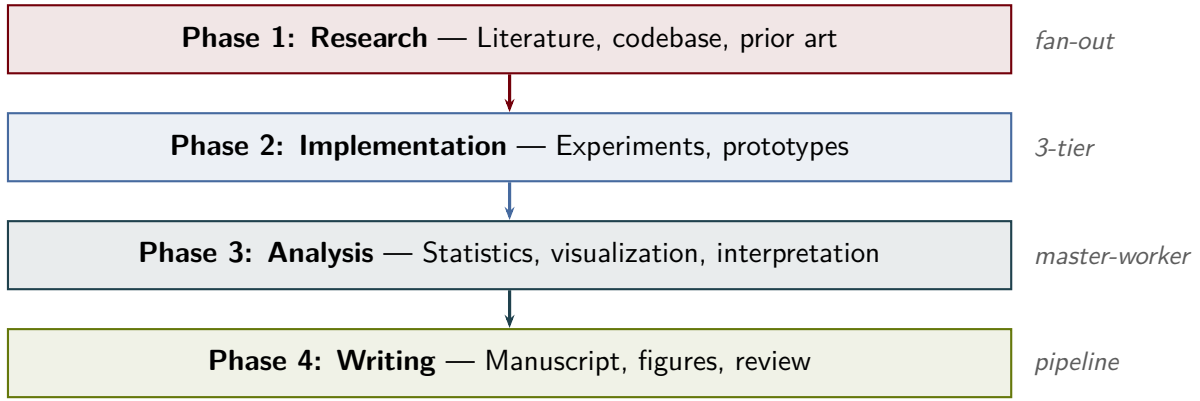


Figure 12: Full research workflow mapped to orchestration patterns. Each phase uses the pattern best suited to its structure.

- File references over content:** Point workers to file paths rather than pasting file contents into prompts.
- Progressive summarization:** When results flow upward, summarize rather than forwarding raw output.

8.3 Batching Strategy

Most systems cap concurrent agents (e.g., 10 in Claude Code). For larger workloads, batch tasks into groups:

$$\text{Batches} = \left\lceil \frac{N_{\text{tasks}}}{N_{\text{max concurrent}}} \right\rceil$$

For 50 tasks with a concurrency limit of 10: 5 sequential batches, each running 10 tasks in parallel. Total time $\approx 5 \times \max(T_{\text{batch}})$, compared to $50 \times T_{\text{task}}$ sequentially.

8.4 State Management

Three approaches to managing shared state across agents:

Centralized. The orchestrator holds all state. Simple but creates a bottleneck.

Distributed. Each agent maintains its own state, synchronized via shared files. Resilient but harder to coordinate.

Hybrid. Critical state (e.g., dependency graph, completion tracking) is centralized; execution state is distributed. This is the recommended approach.

9 Limitations and Open Problems

9.1 Token Cost Multiplication

The primary cost of sub-agent architectures is token consumption. A system with 5 parallel agents uses approximately $5 \times$ the tokens of a single-agent approach. For token-billed APIs, this directly impacts cost.

9.2 Coordination Overhead

Each layer of abstraction adds latency for delegation, result collection, and synthesis. For tasks shorter than ~ 5 minutes, the overhead can exceed the parallelization benefit.

9.3 Context Contamination

When context is passed between agents imprecisely, irrelevant information accumulates. This “context contamination” degrades downstream agent performance. Strict context filtering mitigates but does not eliminate this problem.

9.4 Error Propagation

In deep hierarchies, a failure at a leaf node may not surface until synthesis, by which time recovery is expensive. Robust error handling at every level is essential but adds complexity.

9.5 Reproducibility

LLM outputs are inherently stochastic. Multi-agent systems amplify this variance: the same input

may produce different task decompositions, different worker assignments, and different final outputs across runs. For research applications, this non-determinism must be carefully managed through logging, seed control, and consensus mechanisms.

10 Conclusion

Sub-agent architectures transform single-agent LLM systems into scalable, specialized, parallel workforces. The core insight is simple: *decompose complex problems into scoped tasks, execute them independently, and synthesize the results*. The two-layer pattern handles most use cases; three-tier architectures add a planning layer for complex projects; and hierarchical systems scale to enterprise-grade problems.

For research applications, these patterns map directly onto established workflows: literature review (fan-out/fan-in), experimental execution (three-tier), data analysis (master-worker), and manuscript preparation (pipeline). The result is faster iteration, broader coverage, and higher quality through cross-validation—at the cost of increased token consumption.

As LLM context windows grow and inference costs decline, the economic case for sub-agent architectures will only strengthen. The patterns described here provide a foundation for building effective multi-agent research systems today.